Johnson's algorithm for All-pairs shortest paths

Last Updated : 23 Jul, 2025

The problem is to find the shortest paths between every pair of vertices in a given weighted directed Graph and weights may be negative. We have discussed Floyd Warshall Algorithm for this problem. The time complexity of the Floyd Warshall Algorithm is $\Theta(V^3)$.

Using Johnson's algorithm, we can find all pair shortest paths in $O(V^2 \log V + VE)$ time. Johnson's algorithm uses both Dijkstra and Bellman-Ford as subroutines. If we apply Dijkstra's Single Source shortest path algorithm for every vertex, considering every vertex as the source, we can find all pair shortest paths in O(V*(V + E) * Log V) time.

So using Dijkstra's single-source shortest path seems to be a better option than Floyd Warshall's Algorithm , but the problem with Dijkstra's algorithm is, that it doesn't work for negative weight edge. *The idea of Johnson's algorithm is to re-weight all edges and make them all positive, then apply Dijkstra's algorithm for every vertex.*

How to transform a given graph into a graph with all non-negative weight edges?

One may think of a simple approach of finding the minimum weight edge and adding this weight to all edges. Unfortunately, this doesn't work as there may be a different number of edges in different paths (See this for an example). If there are multiple paths from a vertex u to v, then all paths must be increased by the same amount, so that the shortest path remains the shortest in the transformed graph. The idea of Johnson's algorithm is to assign a weight to every vertex. Let the weight assigned to vertex u be h[u].

We reweight edges using vertex weights. For example, for an edge (u, v) of weight w(u, v), the new weight becomes w(u, v) + h[u] - h[v]. The great thing about this reweighting is, that all set of paths between any two vertices is increased by the same amount and all negative weights become non-negative. Consider any path between two vertices s and t, the weight of every path is increased by h[s] - h[t], and all h[] values of vertices on the path from s to t cancel each other.

How do we calculate h[] values?

Bellman-Ford algorithm is used for this purpose. Following is the complete algorithm. A new vertex is added to the graph and connected to all existing vertices. The shortest distance values from the new vertex to all existing vertices are h[] values.

Algorithm:

1. *Let the given graph be G. Add a new vertex s to the graph, add edges from the new vertex to all vertices of G. Let the modified graph be G'.*
2. *Run the Bellman-Ford algorithm on G' with s as the source. Let the distances calculated by Bellman-Ford be h[0], h[1], .. h[V-1]. If we find a negative weight cycle, then return. Note that the negative weight cycle cannot be created by new vertex s as there is no edge to s. All edges are from s.*
3. *Reweight the edges of the original graph. For each edge (u, v), assign the new weight as "original weight + h[u] - h[v]".*
4. *Remove the added vertex s and run Dijkstra's algorithm for every vertex.*

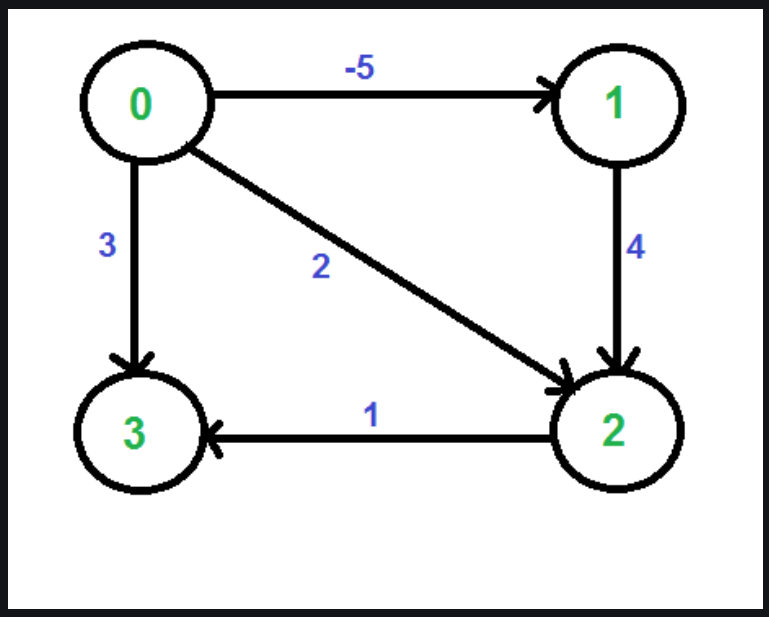
How does the transformation ensure nonnegative weight edges?

The following property is always true about h[] values as they are the shortest distances.

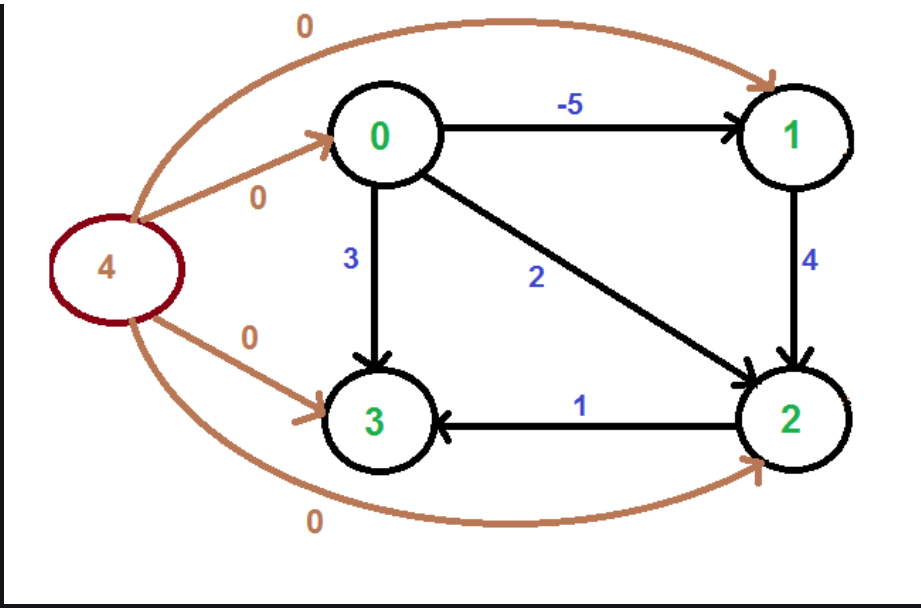
```
h[v] <= h[u] + w(u, v)
```

The property simply means that the shortest distance from s to v must be smaller than or equal to the shortest distance from s to u plus the weight of the edge (u, v). The new weights are w(u, v) + h[u] - h[v]. The value of the new weights must be greater than or equal to zero because of the inequality "h[v] <= h[u] + w(u, v)".

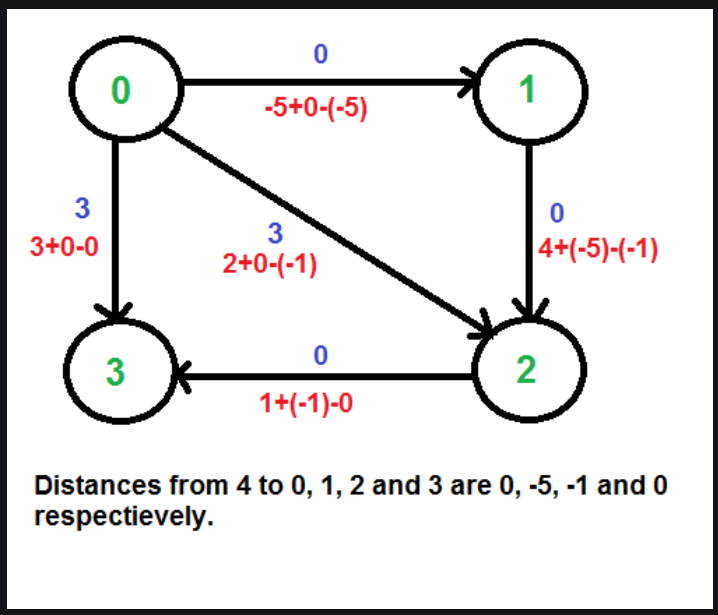
Example: Let us consider the following graph.



We add a source s and add edges from s to all vertices of the original graph. In the following diagram s is 4.



We calculate the shortest distances from 4 to all other vertices using Bellman-Ford algorithm. The shortest distances from 4 to 0, 1, 2 and 3 are 0, -5, -1 and 0 respectively, i.e., h[] = {0, -5, -1, 0}. Once we get these distances, we remove the source vertex 4 and reweight the edges using following formula. w(u, v) = w(u, v) + h[u] - h[v].



Since all weights are positive now, we can run Dijkstra's shortest path algorithm for every vertex as the source.

C++ Java Python JavaScript

```cpp
#include <iostream>
#include <vector>
#include <limits>
#include <algorithm>

#define INF std::numeric_limits<int>::max()

using namespace std;

// Function to find the vertex with the minimum distance
// that has not yet been included in the shortest path tree
int Min_Distance(const vector<int>& dist, const vector<bool>&
visited) {
    int min = INF, min_index;
    for (int v = 0; v < dist.size(); ++v) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

// Function to perform Dijkstra's algorithm on the modified
graph
void Dijkstra_Algorithm(const vector<vector<int>>& graph,
const vector<vector<int>>& altered_graph, int source) {
    int V = graph.size();  // Number of vertices
    vector<int> dist(V, INF);  // Distance from source to each
vertex
    vector<bool> visited(V, false);  // Track visited vertices

    dist[source] = 0;  // Distance to source itself is 0

    // Compute shortest path for all vertices
    for (int count = 0; count < V - 1; ++count) {
        // Select the vertex with the minimum distance that
hasn't been visited
        int u = Min_Distance(dist, visited);
```

```
        visited[u] = true;  // Mark this vertex as visited

        // Update the distance value of the adjacent vertices
of the selected vertex
        for (int v = 0; v < V; ++v) {
            if (!visited[v] && graph[u][v] != 0 && dist[u] !=
INF && dist[u] + altered_graph[u][v] < dist[v]) {
                dist[v] = dist[u] + altered_graph[u][v];
            }
        }
    }

    // Print the shortest distances from the source
    cout << "Shortest Distance from vertex " << source <<
":\n";
    for (int i = 0; i < V; ++i) {
        cout << "Vertex " << i << ": " << (dist[i] == INF ?
"INF" : to_string(dist[i])) << endl;
    }
}

// Function to perform Bellman-Ford algorithm to find shortest
distances
// from a source vertex to all other vertices
vector<int> BellmanFord_Algorithm(const vector<vector<int>>&
edges, int V) {
    vector<int> dist(V + 1, INF);  // Distance from source to
each vertex
    dist[V] = 0;  // Distance to the new source vertex (added
vertex) is 0

    // Add a new source vertex to the graph and connect it to
all original vertices with 0 weight edges
    vector<vector<int>> edges_with_extra(edges);
    for (int i = 0; i < V; ++i) {
        edges_with_extra.push_back({V, i, 0});
    }

    // Relax all edges |V| - 1 times
    for (int i = 0; i < V; ++i) {
        for (const auto& edge : edges_with_extra) {
```

```
            if (dist[edge[0]] != INF && dist[edge[0]] +
edge[2] < dist[edge[1]]) {
                dist[edge[1]] = dist[edge[0]] + edge[2];
            }
        }
    }
    return vector<int>(dist.begin(), dist.begin() + V);  //
Return distances excluding the new source vertex
}

// Function to implement Johnson's Algorithm
void JohnsonAlgorithm(const vector<vector<int>>& graph) {
    int V = graph.size();  // Number of vertices
    vector<vector<int>> edges;

    // Collect all edges from the graph
    for (int i = 0; i < V; ++i) {
        for (int j = 0; j < V; ++j) {
            if (graph[i][j] != 0) {
                edges.push_back({i, j, graph[i][j]});
            }
        }
    }

    // Get the modified weights from Bellman-Ford algorithm
    vector<int> altered_weights = BellmanFord_Algorithm(edges,
V);
    vector<vector<int>> altered_graph(V, vector<int>(V, 0));

    // Modify the weights of the edges to remove negative
weights
    for (int i = 0; i < V; ++i) {
        for (int j = 0; j < V; ++j) {
            if (graph[i][j] != 0) {
                altered_graph[i][j] = graph[i][j] +
altered_weights[i] - altered_weights[j];
            }
        }
    }

    // Print the modified graph with re-weighted edges
```

```
    cout << "Modified Graph:\n";
    for (const auto& row : altered_graph) {
        for (int weight : row) {
            cout << weight << ' ';
        }
        cout << endl;
    }

    // Run Dijkstra's algorithm for every vertex as the source
    for (int source = 0; source < V; ++source) {
        cout << "\nShortest Distance with vertex " << source
<< " as the source:\n";
        Dijkstra Algorithm(graph, altered_graph, source);
```



```
}

// Main function to test the Johnson's Algorithm
implementation
int main() {
    // Define a graph with possible negative weights
    vector<vector<int>> graph = {
        {0, -5, 2, 3},
        {0, 0, 4, 0},
        {0, 0, 0, 1},
        {0, 0, 0, 0}
    };

    // Execute Johnson's Algorithm
    JohnsonAlgorithm(graph);
    return 0;
}
```

Output

```
Following matrix shows the shortest distances between every pair of
vertices
      0      5      8      9
    INF      0      3      4
```

```
    INF     INF       0       1
    INF     INF     INF       0
```

Time Complexity: The main steps in the algorithm are Bellman-Ford Algorithm called once and Dijkstra called V times. Time complexity of Bellman Ford is O(VE) and time complexity of Dijkstra is O((V + E)Log V). So overall time complexity is $O(V^2 \log V + VE)$.

The time complexity of Johnson's algorithm becomes the same as Floyd Warshall's Algorithm when the graph is complete (For a complete graph $E = O(V^2)$). But for sparse graphs, the algorithm performs much better than Floyd Warshall's Algorithm.

Auxiliary Space: $O(V^2)$

Comment
K kartik + Follow 35

Article Tags : Graph DSA Shortest Path

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play
Download on the App Store

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Campus Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
IBM Certification
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved